Wireless Intrusion Detection System for Wi-Fi Deauthentication Attacks

A machine-learning prototype using AWID3 wireless traffic

Project repository: https://github.com/kakeetopius/wids

Abstract

This report presents a wireless intrusion detection system (WIDS) prototype for identifying Wi-Fi deauthentication traffic. The system uses labelled traffic from the AWID3 dataset, extracts frame, radio and traffic-rate features, and classifies observations with a scikit-learn Random Forest. To make the evaluation more representative of deployment on unseen captures, complete capture files are separated into training, validation and final test groups. On the held-out test captures, the system achieved 98.14% accuracy, 99.99% precision and 96.51% recall for deauthentication traffic. The resulting prototype provides confidence-ranked alerts through a small web dashboard. It is intentionally an offline WIDS and does not transmit attack traffic, capture live networks, or take prevention actions.

1. Introduction

Wireless networks depend on management frames to establish and maintain client connections. Deauthentication frames can be abused to disconnect clients repeatedly, making deauthentication flooding a suitable attack class for a small intrusion-detection prototype. A detector should identify suspicious activity without relying on a single field that simply reveals the label, and it should provide an evaluation on traffic that was not used to train the model.

The aim of this project was to build a reproducible WIDS prototype that distinguishes normal traffic from deauthentication traffic, reports measurable classification performance, and presents the result in a form that can be inspected during assessment. The project is a detection system, not a wireless prevention system.

2. Dataset and feature preparation

The project uses the deauthentication portion of the AWID3 wireless intrusion-detection dataset. The complete raw dataset is not included in the repository because of its size and distribution requirements. The supplied model artifact contains the trained pipeline and its evaluation metadata, allowing the dashboard to run without downloading the raw captures.

The selected input variables describe frame characteristics, radio conditions and local traffic volume. They include frame length, time delta, channel frequency, data rate, duration, frame flags, reason code, received signal strength, and one-second frame counts for all traffic, each source and each destination.

The fields `wlan.fc.type` and `wlan.fc.subtype` were deliberately excluded. In this subset they are closely associated with the deauthentication label and could provide an unrealistically direct shortcut for the classifier. Traffic rates are calculated from timestamps and MAC-address fields without using the label.

3. Methodology

3.1 Data partitioning

Rows from each complete capture file remain together during partitioning. Files 25 to 28 are used for validation and files 29 to 32 form the final test set. The remaining files are used for training. The validation set is used only to select the alert threshold; the final test set is reserved for the reported results. This arrangement reduces the risk that near-duplicate frames from one capture appear in both training and evaluation.

Partition	Capture files	Function
Training	Files outside 25–32	Fit the model
Validation	Deauth_25.csv to Deauth_28.csv	Select the decision threshold
Final test	Deauth_29.csv to Deauth_32.csv	Measure generalisation

3.2 Classifier

The classifier is a Random Forest with 300 trees, balanced class weights, a minimum leaf size of two and a fixed random seed. Missing numeric values are replaced with the median using a scikit-learn pipeline. The model produces a probability for the deauthentication class. A threshold of 0.35 was selected on the validation captures to favour detection recall when candidate thresholds produced similar F1 scores.

3.3 Alert presentation

Test-set observations classified as deauthentication traffic are stored as offline replay alerts. The dashboard ranks the displayed alerts by model confidence and shows the capture name, source, destination, timestamp and severity. This presentation is intended to make the model output understandable rather than to imply that the prototype is monitoring a live network.

4. Implementation

The training pipeline is implemented in `src/wids/train.py`. The resulting Joblib artifact is stored at `models/deauth_model.joblib`. A FastAPI application loads this artifact and provides pages for the overview, dataset, model evaluation and detections. It also exposes machine-readable endpoints for health, evaluation metadata and replay alerts.

The system reports its operating state explicitly. The health endpoint identifies the current mode as ready for offline replay, indicates that live capture is disabled, and indicates that response actions are disabled. This prevents the dashboard from presenting stored results as live protection.

5. Results

The final test set contains 33,869 labelled observations: 15,844 normal observations and 18,025 deauthentication observations. The results below were calculated after the threshold had been selected using the validation set.

Measure	**Value**
Accuracy	98.14%
Deauthentication precision	99.99%
Deauthentication recall	96.51%
Deauthentication F1-score	98.22%
Decision threshold	0.35

5.1 Confusion matrix

Actual / predicted	**Normal**	**Deauth**
Normal	15,843	1
Deauth	629	17,396

The matrix shows one normal observation incorrectly raised as an alert. There are 629 deauthentication observations classified as normal. Thus, the principal remaining error in this test is missed attack traffic rather than a high number of false alarms. The recall result is therefore more informative for this use case than accuracy alone.

6. Discussion

The results indicate that the selected features contain useful information for separating the two classes in the AWID3 captures. The very high precision means that an alert in this evaluation is usually associated with deauthentication traffic. The lower recall shows that the detector does not identify every attack-labelled observation, so it should not be described as complete protection.

The capture-file split is a stronger test than a purely random row split, but it does not establish performance in every wireless environment. Radio conditions, access-point configuration, client behaviour and attack generation methods can differ substantially between locations. The reported metrics should therefore be understood as results for this AWID3-based experiment.

7. Safety, ethics and limitations

The prototype is a WIDS rather than a WIPS. It does not generate deauthentication traffic, capture traffic from a live network, block addresses, or alter network configuration. All displayed detections are derived from stored test-set observations. Any future monitor-mode collection or attack-generation experiment must be performed only on an authorised isolated laboratory network.

The main limitations are the use of one public dataset subset, the focus on one attack class, and the absence of live-stream latency measurements. Future work should evaluate independent captures, monitor changes in data distribution, calibrate probabilities, compare a simpler baseline, and extend the approach to other wireless attack classes. Live ingestion would require additional validation and explicit safety controls.

8. Operational verification

The supplied repository can be installed with `uv sync` and started with `uv run uvicorn wids.web.app:app --host 127.0.0.1 --port 8000`. Opening the model page verifies the reported metrics and confusion matrix; opening the detections page verifies the stored alert replay; and the health endpoint verifies that live capture and automatic response are disabled. These checks support the live presentation of the prototype but are not substitutes for the final test-set evaluation.

9. Conclusion

This project demonstrates a complete, safe WIDS workflow for deauthentication-flood detection. It combines leakage-aware feature selection, capture-file holdout evaluation, a ready-made Random Forest, measurable results and an inspectable dashboard. The system detects the required attack class and raises alerts, while its offline-only boundary keeps the demonstration within an authorised and controlled setting.

10. AI assistance

AI coding assistants were used during repository inspection, code refactoring, dashboard integration and preparation of documentation and test ideas. The project author reviewed the implementation, dataset split, reported metrics and safety decisions.